基于 AI 的 5G 网络关键性能指标预测与异常检测系统

系统使用手册

离线数据处理 · 模型训练与评估 · KPI 预测 · 异常检测

适用环境：Windows / Python / Streamlit
数据来源：gotsf-ds / netop/gotsf-ds
源码依据：GitHub main · USER_GUIDE.md
文档版本：2026 年 10 月

使用手册目录

1. 这套系统能做什么

2. 运行环境与启动

3. 项目目录与哪些文件会上传 GitHub

4. 数据获取、审计、预处理

5. 传统机器学习 KPI 预测

6. 深度学习 KPI 预测

7. 异常检测与初步关联分析

8. Streamlit 六个功能菜单

9. 实际调用推理与下载 CSV

10. 实时训练终端、结果归档与常见错误

阅读说明

第 1～4 章介绍环境、数据与目录；第 5～7 章介绍预测模型训练和异常检测；第 8～10 章介绍六模块 Web 操作、CSV 推理和故障排查。运行命令均以项目根目录为当前工作目录。

1. 这套系统能做什么

系统使用公开 netop/gotsf-ds 数据集中的四项 KPI：

字段	解释	注意
DLPRB	下行物理资源块相关指标	保留数据集原尺度
DLThpVol	下行流量体积相关指标	不擅自标注为 Mbps
DLThpTime	下行传输时间相关指标	保留数据集原尺度
MR_number	测量报告数相关指标	保留数据集原尺度

主要有两条独立任务：一是用传统 ML 或 LSTM/GRU/Transformer 预测下一小时 KPI；二是用 3σ、IQR、Isolation Forest、One-Class SVM 标记异常候选，再做 KPI 关联的探索性分析。两条流程都通过 Streamlit 操作。检测结果为需要进一步排查的异常候选。

应用按「数据处理 → 训练与测试评估 → 版本管理 → KPI 实际预测 / 异常实际检测 → 关联分析」组织。模型训练结束后可在同页查看该实验的测试指标和真实值/预测值曲线；「模型管理与实验记录」保留独立保存的历史正式成绩，并允许查看任一独立训练 run_id。用户在「KPI 预测」或「异常检测」上传或生成 CSV 后，调用选中的实际模型权重完成推理。六模块新版界面仍需本机回归测试。

2. 运行环境与启动

打开 VS Code，将仓库根目录设为当前工作目录，保留已有 .venv、data/ 和 resources/。PowerShell 中若已显示 (.venv)，可以直接用 python；不确定时显示调用现有环境：

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pip check
.\.venv\Scripts\python.exe -m streamlit run app.py
```

在终端给出的 Local URL 打开页面，通常是 http://localhost:8501。Streamlit 端口被占用时，以实际显示为准。

项目声明支持 Python 3.10+，但具体 PyTorch、XGBoost、LightGBM、Plotly、websockets 的可用性仍取决于本机版本。

GPU 检查：

```
python -c "import torch; print('PyTorch:',torch.__version__); print('CUDA:',torch.cuda.is_available());
print('GPU:',torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'CPU')"
```

训练设备以 PyTorch 检测结果和实际运行日志为准。

3. 项目目录与哪些文件会上传 GitHub

```
项目根目录/
├── app.py                  Streamlit 入口
├── src/
│   ├── core/               KPI 字段、通用路径、训练参数
│   ├── data/               下载、原始数据审计、预处理、特征工程
│   ├── models/             传统 ML、DL、异常检测、推理
│   ├── analysis/           模型比较、异常统计、鲁棒性检查
│   ├── services/           后台任务、WebSocket、权重版本管理
│   └── ui/                 Web 工作流与结果可视化
├── tests/                  结构、参数、编码、验收数据测试
├── test_inference.py       预测和异常推理合成数据测试
├── test_model_registry.py  本地模型版本发现/归档测试
├── data/raw/               下载后的源 CSV，Git 忽略
├── data/processed/         预处理 CSV、特征缓存，Git 忽略
```

```
├── models/                  未版本化权重，Git 忽略
├── resources/
│   ├── experiments/<run_id>/    ML、DL、异常检测的独立实验结果
│   ├── models/<run_id>/         已发布的权重及元数据
│   ├── jobs/                任务状态、日志、WebSocket 信息
│   └── demo_inputs/             Web 验收输入 CSV
├── reports/                 已追踪的历史实验报告/CSV
├── figures/                 已追踪的历史图表
├── docs/                    技术报告、架构与参数说明
└── requirements.txt
```

仓库归档包含源码、文档和正式实验统计。原始数据与模型权重位于受 .gitignore 排除的本地目录；在新机器上可重新下载数据并训练，或迁移可信模型权重与对应元数据。

4. 数据获取、审计、预处理

4.1 Web 方式

进入「数据管理与预处理」：界面会分别显示 data/raw/ 中四份原始 KPI CSV 是否存在。

依次操作「下载原始数据」→「审计并预处理」→「生成传统 ML 特征」。已有有效数据或特征缓存时，可直接进入后续模块。

4.2 命令行方式

从项目根目录运行：

```
python -m src.data.download_kpi_data
python -m src.data.audit_kpi_data
python -m src.data.preprocess_kpi_data
python -m src.data.build_ml_features
```

数据获取模块使用 netop/gotsf-ds 中固定的四项 KPI 源 CSV，生成以下文件：

操作	本地输入 / 输出
下载	data/raw/DLPRB_train_0w-5w.csv 等四份源文件
审计	reports/raw_data_audit.json、 reports/raw_data_audit_summary.csv
预处理	data/processed/kpi_timeseries.csv、 reports/preprocessing_summary.json
ML 特征	data/processed/ml_features.pkl、 reports/ml_feature_summary.json

历史预处理报告记录 2,880 个波束、840 小时、2,419,200 条观测；训练时间 0–671，测试时间 672–839。StandardScaler 和 MinMaxScaler 参数在训练期间拟合，再应用于测试区间，防止把未来测试分布用于估计训练转换参数。

特征工程使用标准化 KPI 的时刻 t 与滞后 1、2、24、168 小时值，目标是 t+1 的 KPI 原始尺度值。原始时间序列和 ML 特征数据分别保存，便于训练与推理阶段按需读取。

5. 传统机器学习 KPI 预测

在「模型训练」选择「传统 ML」，确认 data/processed/ml_features.pkl 存在后开始训练。脚本包含 Persistence 基线、LinearRegression、DecisionTree、RandomForest、XGBoost、LightGBM。Web 默认勾选快速验收模式，可设置训练/测试抽样上限；取消勾选则执行完整训练和测试。它尚未开放每个算法独立的树深度、树数量等超参数界面。传统 ML Web 快速训练、权重发布和预测 CSV 导出已于 2026-10-23 完成本机验收；本轮没有重跑完整正式训练。

命令行：

```
python -m src.models.train_traditional_models --out resources/experiments/manual_ml_full
```

Web 启动的传统 ML 训练会把模型、指标、预测汇总和图表分别写入 resources/experiments/<run_id>/models/、reports/ 与 figures/，再将成功模型发布到 resources/models/<run_id>/。因此可独立保留已跟踪的正式历史报告。旧命令行调用（不带 `--out`）仍采用固定的 `models/`、`reports/`、`figures/` 路径；若自行从终端训练，请明确指定独立 --out。

传统 ML 本机快速验收命令示例（每次使用新目录）：

```
python -m src.models.train_traditional_models --quick --max-train 512 --max-test 512 --out
resources/experiments/manual_ml_smoke
```

Web 测试推荐默认 512 条训练/测试抽样。quick 模式用于验证功能链路；正式模型性能结论采用仓库保存的完整历史实验数据。

6. 深度学习 KPI 预测

在「模型训练」选 LSTM、GRU 或 Transformer。Web 可配置：

训练参数	Web 初始值	作用
Epochs	12	最大训练轮数
Batch Size	256	每次训练更新的样本量
Learning Rate	0.001	学习率
Hidden Size	32	隐藏层维度
Patience	3	验证损失连续不改善的早停轮数
Sequence Length	24	输入历史小时数
Seasonal Lag	168	季节性参考历史小时数
Seed	42	随机种子
Device	auto	自动选择 CPU/CUDA

高级选项还可更改训练/验证/测试抽样上限。时间切分边界不在 Web 中开放，避免破坏实验可比性。Transformer 当前有四个注意力头，Hidden Size 必须能被 4 整除。

建议初次测试使用 Epochs 3、Batch Size 64、Patience 2，先确认后台和导出链路，再决定是否做正式训练。

命令行示例（每次换一个全新的输出目录）：

```
python -m src.models.train_deep_learning_models --models gru --epochs 3 --batch-size 64 --patience 2 --
out resources/experiments/manual_gru_smoke
```

Web 启动的每次 DL 训练会自动分配 resources/experiments/<run_id>/，将 requested_parameters.json、checkpoint、训练指标和图表保存在独立实验中，并将成功训练的 .pt 与同名 .json 复制到 resources/models/<run_id>/ 供推理。历史正式实验指标继续保存在 reports/w3_deep_learning_metrics.csv；各次新实验在自己的 run_id 目录中独立保存。

历史深度学习指标在 reports/w3_deep_learning_metrics.csv，传统 ML/DL 汇总在 reports/week3/w3_ml_dl_all_metrics.csv、w3_ml_dl_comparison_report.md。训练日志若显示 Early stopping 是正常早停，需再看最终任务状态是否 completed。

7. 异常检测与初步关联分析

在「模型训练」选择「异常检测」，可配置 Isolation Forest 污染率、训练抽样量、One-Class SVM 抽样、最大事件保存数量和随机种子。

脚本使用 3σ、IQR、Isolation Forest、One-Class SVM 标记候选，再按 IF AND (3σ OR IQR) 计算共识。历史完整测试记录中，共识候选 3,165 条，占 483,840 条测试波束小时记录的 0.65%。One-Class SVM 历史结果来自抽样子集，不应与全量测试的检测比例当成相同分母。

命令行建议输出到独立目录：

```
python -m src.models.detect_kpi_anomalies --out resources/experiments/manual_anomaly_smoke
```

Web 的异常训练也会创建 resources/experiments/<run_id>/ 并保存 anomalies.pkl。已有的初步相关性分析报告位于 reports/w3_anomaly_root_cause_preliminary.md 和 reports/w4_anomaly_diagnostics_report.md；统计处理实现于 src/analysis/analyze_anomaly_statistics.py。

当前数据未提供经过验证的故障标签。报告使用异常候选数、标记率和探索性 KPI 相关性描述检测结果。

8. Streamlit 六个功能菜单

菜单	职责
数据管理与预处理	下载公开数据、审计与清洗、生成传统 ML 特征并保存处理产物
模型训练	调整参数、运行训练、查看日志；完成后展示测试指标、真实值与预测值对比曲线及异常统计，按 run_id 保存
模型管理与实验记录	浏览可信模型权重和版本；按 run_id 查看测试集评估结果，或查看固定的历史正式模型比较
KPI 预测	生成演示 CSV 或上传历史 KPI CSV，选择模型，输出下一小时 KPI 预测值、图表与 CSV
异常检测	生成演示 CSV 或上传对齐的全波束数据，选择模型，输出异常候选、逐小时标记率及 CSV
KPI 关联分析	对当前会话异常候选及其原始 KPI 计算相关矩阵，或查看独立异常实验保存的矩阵

训练评估与实际推理：模型训练完成后，系统利用独立测试集的真实值和预测值计算 MAE、RMSE、R^2 并绘制对比曲线。实际上传 CSV 后，推理页面展示对应的下一小时预测结果；测试集评估保存在训练与实验记录页面。

实验版本：历史正式结果位于仓库顶层 reports/；新实验结果位于 resources/experiments/<run_id>/reports/。异常候选标记率属于无监督分析统计量，系统另外保存用于关联分析的样本信息。

9. 实际调用推理与下载 CSV

9.1 生成符合格式的输入文件

已拥有 data/processed/kpi_timeseries.csv 时，直接运行：

```
python -m src.data.prepare_demo_inputs
```

生成两份不会影响原数据的演示输入：

- resources/demo_inputs/forecast_history.csv：一个波束的 169 个连续小时，适合默认传统 ML 与 DL 下一小时预测。

- resources/demo_inputs/anomaly_history.csv：所有波束的最后三个连续小时，第一小时为差分上下文，适合同一数据集训练的异常检测权重。

输入 CSV 必须包含：

```
Time,base_station,cell,beam,DLPRB,DLThpVol,DLThpTime,MR_number
```

预测输入的各波束时间应唯一且连续，ML 需要至少 169 小时历史；DL 输入长度还取决于 checkpoint 侧车 JSON 的 seq_len 和 seasonal_lag。需要更长历史时可在演示数据脚本后附 --forecast-hours 240 等参数。

异常检测必须与模型训练时的全部波束 ID 和排序一致，所有波束有同一条连续时间轴，并至少带一个上下文小时。不能用仅含一个波束的预测 CSV 去测试训练于全体波束的异常模型。

9.2 Web 直接生成、预测与异常检测

1. 进入「KPI 预测」或「异常检测」页面，选择对应的本机模型权重。传统 ML 模型通常输出单项 KPI；LSTM、GRU、Transformer 联合输出四项 KPI。

2. 选择「系统演示数据」，点击「从本机预处理数据生成演示 CSV」，系统将基于 data/processed/kpi_timeseries.csv 创建 resources/demo_inputs/ 中的演示数据；也可选择「上传 CSV」导入自己的合规数据。

3. 点击「预测下一小时 KPI」或「运行异常检测」，查看本次推理结果与图表，并通过页面下载结果 CSV。

4. 完成异常检测后进入「KPI 关联分析」，选择「本次异常检测」。系统按基站、小区、波束和 Time 匹配候选标记与原始 KPI，对共识候选计算 Pearson 相关矩阵；候选少于三条时提示样本不足。

5. 需要查看 MAE、RMSE、R^2 与测试集真实值对比曲线时，进入「模型训练」查看刚完成的实验，或在「模型管理与实验记录」选择对应 run_id。

演示 CSV 使用本机已经预处理的公开数据，适合检验输入、模型回载与可视化流程。异常检测输入应符合训练时的波束集合与连续时间要求。

Web 当前限制上传约 50 MB。模型 .pkl 采用 Python pickle 格式，应当仅加载可信的本地模型文件。

10. 实时训练终端、结果归档与常见错误

训练由 src/services/web_jobs.py 调度，src/services/web_stream_worker.py 通过本机 WebSocket 推送后台 stdout/stderr，页面为只读输出窗。状态文件 resources/jobs/current.json，日志 resources/jobs/current.log，工作器故障诊断 resources/jobs/stream_errors.log。关闭网页不会主动结束后台进程，但不要在训练期间删除项目或覆盖正在执行的脚本。

- `UnicodeEncodeError: 'gbk' codec ... $R^2$`：Windows 进程编码问题。最新代码通过 src/services/process_env.py 强制 UTF-8；请确认已拉取最新分支并重新启动 Web/任务。

- `different train_scale_mean`：把不同标准化协议的 DL 实验混写到同一历史报告目录。Web 已为 DL 自动创建独立输出目录；CLI 必须指定一个未用过的 --out。

- `ModuleNotFoundError: src.train_...`：旧平铺式脚本已删除。改用 python -m src.models.train_...。

- 模型下拉框为空：GitHub 中保存的是报告和统计文件，模型推理需要本机权重；检查本机 models/ 或 resources/models/。DL checkpoint 必须有同名 JSON 元数据。

- CUDA 不可用：核对 torch.cuda.is_available() 与 PyTorch/CUDA 安装，不要仅凭 RTX 显卡存在推断训练正在使用 GPU。

- 模型评估与推理结果的区别：测试集 MAE、RMSE、R^2 与真实值对比曲线位于「模型训练」「模型管理与实验记录」；当前输入 CSV 的下一小时预测结果位于「KPI 预测」。

- WebSocket 断连：先看 stream_errors.log 和 current.log，不要反复启动相同训练任务。

- 无法导出结果：先点击「预测下一小时 KPI」或「运行异常检测」；切换模型权重或 CSV 后重新执行本次推理。